

Anil Seth



Are We Safe Using Apps on Tablets?

How do various open source mobile platforms protect users from malicious applications?

Recently, I learnt of the availability of Android on EeePC. I dusted off my old EeePC 701 and installed Android 4 from <http://www.android-x86.org>. It was straightforward—Wi-Fi and the function keys I tried, worked. I was actually curious to see how usable Android would be with a mouse and keyboard. I expected that the absence of a touch screen would make it irritating, but I was pleasantly surprised. I found myself absent-mindedly clicking and dragging a Web page to scroll even on the desktop! It may indeed be better to find an alternate way to select text and use click-and-drag for scrolling the screen's visible area. Multi-touch on the touch-pad would make even more touch-screen gestures, like pinch to shrink or zoom, available without a touch-screen.

However, my exploration of Android and similar environments changed once I visited the applications marketplace. Searching for a Web browser did not show Firefox (it's not compatible with EeePC, and thus did not show up), but lots of other applications. The options for narrowing down the search were only free and paid. There was no separate grouping for open source. The reason for my concern was that before installing an application, the marketplace cautions us about the resources and facilities an application would use, and prompts us to think about whether we wish to allow that. My anxiety was about how I would know that I could trust an application?

In desktop environments, rogue applications use system flaws to access information. However, if a user is consciously running an application, nothing prevents the application from accessing the contents of files owned by the user—unless, of course, the user is using SELinux and has configured it suitably for resources used by each application.

Many of us would avoid running a commercial or closed-source application unless we trusted the source, or have no easy alternate—for example, Flash. I am inclined to trust an open source application, believing that it is not

likely to be misleading, whereas I would be concerned about the motivation of the group offering a free, but not open source, application.

On the smartphone and tablet platforms, the number of applications available has given them bragging rights, as if the absurd number of applications for a platform makes it more usable! (I would urge people to see the talks by Barry Schwartz and Sheena Iyengar on ted.com!) Instead, I was struck by indecision (more like paralysis) because I was not sure if, by allowing an application access to the disk and network, I was risking exposing my personal information to dubious application makers. Could the application read the user names and passwords stored by the Web browser for various sites? And this was on a system I was using to just test and explore Android!

Android is based on Linux, which is secure—but not against the ignorance of users (or perhaps, the politically correct term is 'social engineering'). So, I wondered how Android was handling these concerns, which led me to <http://source.android.com/tech/security/>

Android—each application is a user

The Android platform takes advantage of the security inherent in Linux by assigning a unique user ID to each Android application. Linux ensures that the resources used by each application are isolated from the others. Hence, permission to access the disk allows it to use the disk, but does not permit it to read any other file of any other application, as that is owned by a different user—i.e., another application, in this context! So, an application is running in a sandbox using the standard capabilities of the Linux kernel. It is a remarkably simple method, and has been known to work based on years of experience on desktops and servers.

Applications may need to share data, and may do so using the standard Linux mechanisms, subject to the security policies. However, Android includes a new IPC mechanism, which is managed by the Android